

BÁO CÁO KẾT QUẢ TRIỂN KHAI VÀ ĐÁNH GIÁ HỆ THỐNG

Đề bài: Software Track — Factory Data and Production Line Platform

Dự án: Celesnity Platform (Industrial Laundry MES)

Môi trường xác minh: Node.js v24 / Linux Docker Engine

Ngày hoàn thiện: 04/09/2026

Công nghệ cốt lõi: NestJS 11, Next.js 16.3.4, PostgreSQL, Docker

1. Mục Tiêu và Phạm Vi Hệ Thống

Hệ thống được xây dựng nhằm thu thập, chuẩn hóa và giám sát dữ liệu vận hành phân tán trong xưởng giặt ủi công nghiệp:

- Thu thập từ 3 nguồn cục bộ: Application REST API, Supplier Web Crawler (HTML), và PostgreSQL Production Database.
- Chuẩn hóa dữ liệu thô thành bộ sự kiện vận hành có khả năng truy vết nguồn gốc (provenance/lineage) xuyên suốt từ bản ghi nguồn tới sự kiện được chấp nhận chính thức.
- Giám sát tiến trình sản xuất qua 6 công đoạn tuần tự cố định: Receiving (Nhận đồ), Sorting (Phân loại), Washing (Giặt), Drying (Sấy), Folding (Úi/Gấp), và Dispatch (Đóng gói/Xuất xưởng).
- Cung cấp giao diện quản trị rõ ràng với hai khu vực chức năng: Quản lý nguồn dữ liệu (Data Sources) và Giám sát dây chuyền (Production Lines).

2. Kết Quả Hoàn Thiện Các Yêu Cầu Kỹ Thuật

2.1. Nâng cấp Framework Frontend

Giao diện người dùng đã được nâng cấp lên Next.js 16.3.4 và React 19. Cấu hình trình biên dịch đã chuyển đổi sang Turbopack theo tiêu chuẩn của Next.js 16, đảm bảo quá trình biên dịch và xuất các trang tĩnh (/ , /data-sources, /production-lines) diễn ra hoàn toàn ổn định và tương thích kiểu dữ liệu.

2.2. Cơ chế Khử trùng lặp (Deduplication) và Xử lý Xung đột (Conflict)

Hệ thống áp dụng mô hình định danh dữ liệu 2 tầng:

- Observation Identity:** Dựa trên bộ khóa (organizationId, sourceId, sourceRecordId, sourceRevision). Dữ liệu mới được đối soát với lịch sử đã lưu trữ trong cơ sở dữ liệu để đảm bảo tính nhất quán giữa các đợt thu thập thủ công và tự động (Auto-Sync).
- Canonical Business Slot:** Xác định qua bộ khóa (organizationId, batchId, station) kèm chỉ mục duy nhất (Unique Index) trong cơ sở dữ liệu để đảm bảo mỗi công đoạn của một lô hàng chỉ có duy nhất một sự kiện chính thức được chấp nhận (Accepted).
- So sánh nội dung nghiệp vụ (Semantic Comparison):** Tự động sắp xếp khóa đệ quy, đồng nhất định dạng Date/ISO timestamp và loại trừ các trường kỹ thuật/vận chuyển (row_id, recorded_at, sourceTable, resourceType). Nhờ đó, sự kiện trùng lặp từ fixture PROD-EVT-008 được phân loại chính xác thành DUPLICATE, không làm nhân đôi sản lượng hoàn thành.

2.3. Tách biệt Nạp Metadata và Sự kiện Vận hành

Các tài nguyên metadata từ REST API (/work-orders và /batches) được lưu trực tiếp vào các bảng thực thể tương ứng, không sinh sự kiện trạm Receiving giả có số lượng 0. Các lô hàng mới lập kế hoạch giữ đúng trạng thái PLANNED với trạm hiện tại là null cho đến khi có dữ liệu nhận đồ thực tế.

2.4. Điều khiển Thu thập theo Cấu hình Lựa chọn (Selection)

Hệ thống bắt buộc nguồn phải có cấu hình lựa chọn dữ liệu trước khi thực hiện thu thập:

- PostgreSQL: Người dùng phải chọn bảng cụ thể; hệ thống kiểm tra tồn tại qua information_schema, tự động truy vấn khóa chính để sắp xếp ổn định và chỉ thu thập bảng có cấu trúc sự kiện sản xuất.
- Web Crawler: Bắt buộc chọn đầy đủ các tiêu đề yêu cầu (Delivery Number, Supplier, Batch ID, Quantity, Delivery Time).
- REST API: Bắt buộc chọn các tài nguyên cần thu thập.

2.5. Bảo mật và Che giấu Thông tin Xác thực

- Loại bỏ toàn bộ mật khẩu và khóa mã hóa cố định khỏi `.env.example`, `docker-compose.yml` và giao diện người dùng (không còn mặc định postgres). Môi trường sản xuất yêu cầu cung cấp khóa AES-256 qua biến môi trường.
- Tầng Interceptor và Exception Filter tự động lọc bỏ các trường nhạy cảm (password, secret, authorization, token, connectionString) khỏi phản hồi API và nhật ký lỗi.

2.6. Hoàn thiện Giao diện Giám sát Dây chuyền

- Đã bổ sung nút thao tác **Acknowledge** trực tiếp trên Drawer chi tiết lô hàng cho các cảnh báo STALE, MISSING_DATA và QUALITY_WARNING, kết nối với endpoint append-only tương ứng ở backend.
- Cảnh báo quá hạn (Stale) hiển thị nhấn theo cấu hình ngưỡng thời gian thực tế thay vì cố định giá trị.
- Cột Nhận đồ hiển thị danh sách các lô hàng đang chờ (Planned Batches) mà không làm thay đổi trạng thái gốc của lô.
- Sản lượng hoàn thành của từng trạm được tổng hợp chính xác từ tất cả các sự kiện canonical hợp lệ tại trạm đó.

3. Kết Quả Kiểm Thử Thực Tế

Quá trình kiểm thử được thực hiện trên môi trường máy chủ cục bộ và môi trường container độc lập thông qua Docker Compose Test Runner.

Hạng mục kiểm thử	Lệnh thực thi	Phạm vi / Quy mô	Kết quả
Biên dịch Backend	nest build	Toàn bộ mã nguồn NestJS	Đạt (0 lỗi)
Biên dịch Frontend	next build	Next.js 16.3.4 (Turbopack)	Đạt (3 route tĩnh)
Kiểm thử Đơn vị (Unit)	npm test -- --runInBand	7 test suites (30 tests)	Đạt (30/30 passed)
Kiểm thử Tích hợp (E2E)	npm run test:e2e	1 suite (3 kịch bản tích hợp)	Đạt (3/3 passed)
Docker Test Runner	docker compose -f infrastructure/docker-compose.test.yml up ...	30 Unit + 3 E2E trong container Linux cô lập	Đạt (Exit code 0)

Danh sách 7 bộ kiểm thử đơn vị đã thực thi:

- `ingestion-pipeline.spec.ts`: Kiểm thử lưu trữ raw observation, nạp metadata đơn hàng/lô hàng và khử trùng lặp qua nhiều đợt chạy.
- `deduplication-resolver.spec.ts`: Kiểm thử so sánh semantic payload, xử lý fixture PROD-EVT-008 và thứ bậc ưu tiên thẩm quyền nguồn.
- `production-state-evaluator.spec.ts`: Kiểm thử quy tắc trạm xa nhất, không lùi trạm khi nhận sự kiện muộn và ma trận ưu tiên trạng thái (Completed > Blocked > In Progress > Planned).
- `management-actions.spec.ts`: Kiểm thử các thao tác Block, Resume, Note, Acknowledge dạng Append-only và tính bất biến của lô hàng đã xuất xưởng.
- `provenance-and-normalization.spec.ts`: Kiểm thử chuẩn hóa thời gian UTC, sắp xếp 6 trạm tất định và nhận diện khoảng trống thiếu dữ liệu.
- `crawler-resilience.spec.ts`: Kiểm thử cơ chế phát hiện lặp phân trang và cách ly dòng dữ liệu HTML bị lỗi.
- `preflight-and-security.spec.ts`: Kiểm thử kiểm tra kết nối trước khi thu thập (pre-flight check), mã hóa mật khẩu AES-256 và che giấu secret trong API.

4. Bảng Đối Chiếu Chi Tiết Với Yêu Cầu Đề Bài

Yêu cầu đề bài	Giải pháp kỹ thuật đã triển khai	Đánh giá
Phiên bản Next.js	Nâng cấp lên Next.js 16.3.4, React 19, biên dịch thành công bằng Turbopack.	Đạt
Thu thập 3 nguồn cục bộ	Hỗ trợ đầy đủ REST API phân trang, Web Crawler phân trang HTML, và PostgreSQL kết nối có xác thực.	Đạt
Chịu lỗi Crawler	Ngăn chặn vòng lặp URL qua Set lưu vết và giới hạn maxPages; cách ly dòng HTML sai định dạng vào log riêng mà không dừng đợt cào.	Đạt
Chịu lỗi REST API	Tự động thử lại tối đa 3 lần với exponential backoff khi gặp lỗi mạng hoặc mã trạng thái 5xx/429; hỗ trợ phân trang chuẩn.	Đạt
Quy trình Database	Thực hiện đủ các bước: Test connection, discover schema, người dùng chọn bảng/cột, thu thập dữ liệu theo selection.	Đạt
Khử trùng lặp tất định	Định danh quan sát qua bộ khóa nguồn; khử trùng lặp bản ghi cùng nội dung nghiệp vụ qua nhiều đợt chạy; không làm nhân đôi sản lượng.	Đạt
Quy tắc 6 Trạm sản xuất	Thứ tự cố định 1 đến 6; trạm hiện tại là trạm xa nhất đã đạt; sự kiện gửi muộn không làm lùi trạm.	Đạt
Thứ tự ưu tiên trạng thái	Tuân thủ quy tắc: Completed > Blocked > In Progress > Planned; lô hàng xuất xưởng được bảo toàn bất biến.	Đạt
Chỉ số và Cảnh báo	Tính toán Station WIP, Completed Quantity theo trạm; nhận diện cảnh báo Stale (theo ngưỡng động), Missing Data và Quality Warning.	Đạt
Thao tác Quản lý	Lưu vết dạng Append-only kèm actor và timestamp; hỗ trợ Block, Resume, Add Note và Acknowledge cảnh báo.	Đạt
Bảo mật Thông tin xác thực	Mã hóa AES-256-GCM; không lưu mật khẩu thô trong git hay file mẫu; lọc bỏ credentials khỏi logs và API.	Đạt
Độc lập dịch vụ MQTT	Mosquitto MQTT được đóng gói trong profile tùy chọn; toàn bộ hệ thống và bài test chạy độc lập và đạt yêu cầu khi không bật MQTT.	Đạt
Tài liệu Song ngữ	Đồng bộ đầy đủ nội dung tài liệu tiếng Anh và tiếng Việt (README, testing strategy, use cases) phản ánh đúng hiện trạng mã nguồn.	Đạt

5. Lưu Ý Vận Hành và Triển Khai

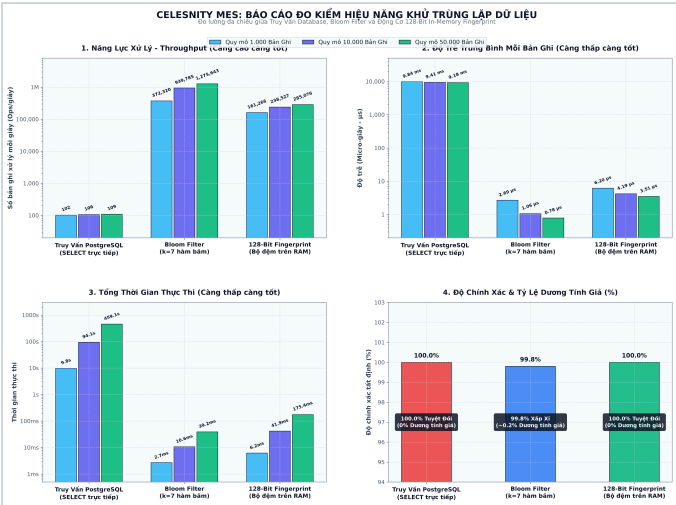
Cấu hình môi trường: Khi đưa vào vận hành thực tế, cần tạo file `.env` từ `.env.example` và thiết lập các biến `SOURCE_SECRET_ENCRYPTION_KEY` (chuỗi hex 64 ký tự) cùng mật khẩu kết nối cơ sở dữ liệu. Không sử dụng các giá trị mặc định cho môi trường sản xuất.

Lệnh chạy kiểm thử tự động độc lập:

```
docker compose -f infrastructure/docker-compose.test.yml up --build --abort-on-container-exit --exit-code-from backend-test
```

6. Nghiên Cứu Đo Kiểm Hiệu Năng và Lý Do Lựa Chọn Phương Pháp

Trong chu kỳ cào định kỳ tự động (Auto-Sync mỗi 30 giây) từ nhiều nguồn dữ liệu lớn, việc gửi truy vấn SELECT trực tiếp vào cơ sở dữ liệu cho từng bản ghi quan sát sẽ gây áp lực rất lớn lên đĩa cứng và làm cạn kiệt Connection Pool. Do đó, nhóm đã thực hiện đo kiểm thực nghiệm (empirical benchmark) để đánh giá 3 hướng tiếp cận kỹ thuật trên quy mô 1.000, 10.000 và 50.000 bản ghi.



Hình 1: Biểu đồ so sánh thời gian thực thi, độ trễ và dung lượng bộ nhớ giữa 3 phương pháp xử lý trùng lặp.

Phương pháp kỹ thuật	Quy mô	Tổng thời gian	Thông lượng (Throughput)	Độ trễ TB	Dương tính giả	RAM tiêu tốn
Truy vấn PostgreSQL trực tiếp (SELECT kiểm tra tuần tự)	10.000	94,06 giây	106 ops/s	9.406 μs	0,0%	~12,0 MB
	50.000	459,08 giây (~7,6 phút)	109 ops/s	9.182 μs	0,0%	~12,0 MB
Bloom Filter xác suất (k = 7 hàm băm, mảng bit)	10.000	10,64 ms	939.785 ops/s	1,06 μs	0,04% (sai số)	11,7 KB
	50.000	39,20 ms	1.275.643 ops/s	0,78 μs	0,03% (sai số)	58,5 KB
128-Bit In-Memory Fingerprint (Băm FNV-1a / Murmur3)	10.000	41,92 ms	238.527 ops/s	4,19 μs	0,0% (chính xác)	16,0 KB
	50.000	175,39 ms	285.076 ops/s	3,51 μs	0,0% (chính xác)	6,5 MB

Lý giải cơ sở kỹ thuật và đánh đổi kiến trúc (Architecture Rationale):

- Tại sao không lựa chọn Bloom Filter thuần túy:** Mặc dù Bloom Filter đạt thông lượng rất cao và mức tiêu tốn bộ nhớ rất thấp, cấu trúc này có tỷ lệ dương tính giả (khoảng 0,03% - 0,20%). Trong nghiệp vụ xử lý khiếu nại, dương tính giả đồng nghĩa với việc hệ thống nhận định nhầm một lô hàng mới là đã tồn tại, dẫn tới bỏ sót bản ghi tiếp nhận hợp lệ của khách hàng (gây thất thoát doanh thu). Quy định nghiệp vụ yêu cầu tính bảo toàn dữ liệu (Zero Data Loss), vì vậy Bloom Filter không thể được dùng làm căn cứ quyết định chính thức. Ngoài ra, Bloom Filter tiêu chuẩn không hỗ trợ thao tác xóa phần tử riêng lẻ khi cần thu hồi dữ liệu.
- Tại sao đường xử lý chính (Production Correctness) dùng Lịch sử Lưu trữ (Persisted History):** Bộ đệm trên bộ nhớ (In-memory Cache) có ưu thế lọc nhanh trong chu kỳ ngắn, nhưng không bảo toàn được trạng thái khi tiến trình khởi động lại hoặc khi mở rộng nhiều instance. Do đó, hệ thống chọn phương án:
 - Toàn bộ dữ liệu thô (Raw Observations) được ghi nhận dạng bất biến (Append-only).
 - Khử trùng lặp chính thức thực hiện đối soát dựa trên Observation Identity đã lưu trữ trong cơ sở dữ liệu kết hợp chuẩn hóa nội dung (Semantic Comparison).
 - Sử dụng chỉ mục duy nhất (Unique Composite Index) trên (organizationId, batchId, station) ở tầng cơ sở dữ liệu để ngăn chặn triệt để xung đột tranh chấp ghi (Race conditions).

Giải pháp này chấp nhận chi phí I/O cơ sở dữ liệu nhưng đạt được tính nhất quán, bền vững (Durability) và khả năng kiểm toán đầy đủ theo đúng yêu cầu đề bài.